

# SAP ABAP AMDP / SQLScript / Code-to-Data 課程

REST 課程 ( [src/ABAP\\_Training\\_REST/](#) · rs01~rs11 ) 之後的下一階段候選之一。課綱草案 ( 2026-07-19 ) · am01 ~ am06 已出題 ( 2026-07-19 · am06 的 AMDP Debugger 操作待使用者於 Eclipse 實際確認 ) · 其餘題目待逐題出題。

## 課程定位

- **對象**：完成 OOP / REST 課程的學員 ( 會 Class/Interface 、 Open SQL 、 BAPI 呼叫 )
- **技術範圍**：AMDP ( ABAP Managed Database Procedure ) + SQLScript 語法本身 + Code-to-Data ( 把邏輯下推到資料庫層 ) 的觀念，範圍限定在 AMDP 這條線，**不含** RAP/CDS 那條現代化路線的其餘主題 ( Behavior Definition 、 Service Binding 等留待另一門課 )
- **前置知識缺口**：前面所有課程 ( 基礎課/OOP/REST ) 用的都是 Open SQL——ABAP 應用層迴圈 + SQL 讀寫；AMDP 是完全不同的典範 ( 邏輯本身跑在資料庫引擎裡 )，SQLScript 又是一個新語法，所以這門課要花比 REST 課更多篇幅在「語法本身」，不能只教「怎麼包一個 AMDP Method」
- **系統確認**：目前連線系統是 On-Premise S/4HANA 1909 ( [SAP\\_BASIS 754](#) )，資料庫是 **SAP HANA 2.0 SPS04** ( [DBSystem=HDB](#) )——AMDP 硬性要求 HANA 資料庫，這套系統符合；ADT Discovery 已確認有 [AMDP Debugger](#)、[Data Preview for AMDP](#)、[ABAP Database Procedure Proxies](#) 等工具鏈，可以在這套系統直接開發與偵錯，不需要額外環境
- **結業標準 ( 草案 )**：能判斷一段邏輯該留在 ABAP 應用層還是下推到 AMDP、能獨立寫一個中等複雜度的 SQLScript 程序 ( 含變數、流程控制、多表 JOIN/聚合 )、知道怎麼從 ABAP 呼叫並處理例外、能把 AMDP 包成 CDS Table Function 供 CDS View 使用、**能寫一個同時 EXPORTING 兩個 ( 以上 ) Internal Table 的 AMDP Method**——這是 AMDP 跟一般 ABAP Method ( [RETURNING](#) 只能單一值 ) 本質不同的地方，也是 SQLScript 的重要特性，課程至少要有一題明確示範

## 教材慣例 ( 比照 OOP/REST 課程 )

- 每題三件套：題目 [amNN\\_主題.md](#) + PDF 講義 ( [node tools/md2pdf.js](#) [src/ABAP\\_Training\\_AMDP](#) ) + 答案快照
- 每題 md 開頭 ( [## 學習目標](#) 之前 ) 要有 [## Lecture](#) 完整背景知識講解，延續 REST 課程 rs07 起養成的慣例
- 答案物件命名：AMDP 本質上還是 ABAP 類別 + 方法 ( 用 [BY DATABASE PROCEDURE](#) 宣告 )，沿用 [ZCL\\_AMnn\\_\\*](#)；牽涉到 CDS Table Function 的題目另外命名 DDL Source ( 暫定 [Z\\_AM\\_TF\\_nn](#)，實際命名視 CDS 物件慣例確認 )，套件 [\\$TMP](#)
- 資料模型：優先沿用 SCARR/SFLIGHT/SBOOK 航班模型 ( 跟 OOP/REST 一致 )，除非某題需要更大量測試資料才能看出 Code-to-Data 的效能差異，才考慮換模型或造測試資料

## 課綱 ( 草案，待確認與逐題出題 )

| #    | 主題                  | 內容重點                                                                                                                               | 銜接前面課程                                           | 狀態  |
|------|---------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|-----|
| am01 | 為什麼要 Code-to-Data + | Open SQL「搬資料回應層算」vs AMDP「邏輯下推到資料庫算」的本質差異；AMDP = ABAP Method 包一段 SQLScript； <a href="#">IF_AMDP_MARKER_HDB</a> 、 <a href="#">BY</a> | 對照 REST rs01 的破題方式；呼應 RAP/CDS 討論裡「這套系統是 HANA」的前提 | 已驗收 |

| #    | 主題                                | 內容重點                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 銜接前面課程                                                   | 狀態  |
|------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|-----|
|      | AMDP 架構總覽                         | <p><b>DATABASE PROCEDURE FOR HDB</b></p> <p><b>LANGUAGE SQLSCRIPT</b> 語法骨架；實測發現 <b>AMDP</b> 簽章限制 ( 所有參數須 <b>VALUE()</b>、<b>DEFAULT</b> 只能常數不能 <b>sy-mandt</b> ) 與最重要的一課：<b>AMDP</b> 不會像 <b>Open SQL</b> 自動做 <b>Client</b> 過濾，<b>SCARR</b> 在這套系統多 <b>Client</b> 灌了同一批資料，不加 <b>WHERE mandt = :iv_mandt</b> 會撈出重複資料</p>                                                                                                                                                                                                                                                                                                                                                 |                                                          |     |
| am02 | 第一個 AMDP Method：Signature 規則與呼叫方式 | <p>AMDP Method 參數限制 ( 修正：elementary 純量型別跟 Table Type 都可以用，am01/am02 的 <b>iv_mandt/iv_carriid</b> 就是 elementary 純量參數；真正不能用的是「非 Table 包裝的 Structure」；只有 <b>IMPORTING/EXPORTING</b>，沒有 <b>RETURNING/CHANGING</b> )；從一般 ABAP 呼叫 AMDP Method 跟呼叫一般 Method 語法上完全一樣 ( 呼叫端無感 )；同時 <b>EXPORTING</b> 兩個 <b>Internal Table</b> 的範例 ( <b>ZCL_AM02_FLIGHT_STATS</b>：同一次 SQLScript 運算，一次吐出 <b>SFLIGHT</b> 明細 + 依 <b>connid</b> 分組的彙總統計兩張表 )，對照一般 ABAP Method 只能 <b>RETURNING</b> 單一值/表格的限制；呼叫端 <b>ZR_AM02_DEMO</b> 用 <b>cl_salv_table</b> 把這兩張回傳的 <b>Internal Table</b> 各自呈現成一個 <b>ALV</b> ( <b>ALV</b> 需要 GUI，已用 <b>WRITE</b> 版本自行驗證過資料邏輯正確，畫面效果經使用者於 <b>SAP GUI</b> 執行確認無誤 )</p> | 承 OOP op02 方法參數的對照；ALV 呈現承 OOP op11 <b>cl_salv_table</b> | 已驗收 |
| am03 | SQLScript 基本語法：變數、流程控制            | <p><b>ZCL_AM03_PRICE_TIER</b>：DECLARE 純量變數 + <b>DECLARE CURSOR</b>、<b>FOR ... AS &lt;cursor_name&gt; DO ... END FOR</b> 迴圈、<b>IF/ELSEIF/ELSE</b> 分支，逐筆分類票價高低並累計三個等級筆數，對照另一段用宣告式 <b>CASE WHEN</b> 一次 <b>SELECT</b> 做同樣分類的寫法；實測發現 <b>FOR &lt;var&gt; AS SELECT ...</b> ( 直接內嵌查詢 ) 語法不合法，必須先 <b>DECLARE CURSOR</b> 再用游標名稱，已在講義記錄</p>                                                                                                                                                                                                                                                                                                                                    | —                                                        | 已驗收 |
| am04 | SQLScript 集合處理：                   | <b>ZCL_AM04_ROUTE_LOAD</b> ：WITH ... AS (...) CTE 先依航線彙總 <b>SFLIGHT</b> ，再                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 對照 REST rs05 的 WHERE 條件組合手                               | 已驗收 |

| #    | 主題                             | 內容重點                                                                                                                                                                                                                                                                                                         | 銜接前面課程                                                          | 狀態                                                                                    |
|------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------------|
|      | 多表 JOIN / 聚合 / CTE             | JOIN SCARR 補上公司名稱算出每條航線載客率；實測發現兩個坑：AMDP 簽章 USING 子句多個物件要空白分隔、不是逗號；SQLScript 的 JOIN ON 條件必須明寫 MANDT ( 跟 Open SQL JOIN ON 條件不能寫 MANDT 正好相反，兩者都指向「Client 處理自動化在哪一層」這個核心對照 )                                                                                                                                     | 法；呼應<br>.claude/rules/sap-adt-mcp.md 第 10 節<br>Open SQL JOIN 限制 |                                                                                       |
| am05 | 錯誤處理與例外                        | ZCL_AM05_FLIGHT_VALIDATOR :<br>DECLARE ... CONDITION FOR SQL_ERROR_CODE + SIGNAL 主動拋錯，ABAP 端 RAISING cx_amdp_error + TRY...CATCH 攔截；實測發現 SIGNAL 的條件名稱必須先 DECLARE ... CONDITION 宣告，不能直接用 ( 原本以為 SQLSCRIPT_ERROR 是內建可用的名稱 ) ； get_text( ) 拿到的是技術性訊息 ( 含程序名/行號 ) ，自訂文字埋在最後面，不適合直接給使用者看；AMDP 沒辦法呼叫回 ABAP ( 單向限制) | 承 OOP op09 例外類別設計                                               | 已驗收                                                                                   |
| am06 | AMDP 除錯與資料預覽                   | ZR_AM06_DEMO：重用 am04 已驗證的 AMDP 方法，改用經典 ALV ( REUSE_ALV_GRID_DISPLAY Function Module，手動組 IT_FIELDCAT ) 呈現結果，對照 op11/am02 教過的 cl_salv_table ( Functional ALV，靠型別反射自動產生欄位目錄 ) 兩種世代的差異；Eclipse ADT 內建 AMDP Debugger 操作步驟 ( 本題唯一需要使用者在 Eclipse 手動操作，Claude 端無法自動驗證 ) 、Data Preview 快速查資料現況                        | —                                                               | 待使用者確認 ( ALV 邏輯已驗證啟用成功，資料沿用 am04；ALV 畫面與 AMDP Debugger 操作待使用者於 SAP GUI/Eclipse 實際確認 ) |
| am07 | CDS Table Function：AMDP 的另一個身分 | DEFINE TABLE FUNCTION ... AS SELECT FROM 搭配 AMDP 實作、跟一般 CDS View 的差異 ( Table Function 可以塞任意 SQLScript 邏輯，View 只能宣告式查詢 ) 、Association 限制                                                                                                                                                                      | 呼應 RAP/CDS 討論的技術背景                                              | 未出題                                                                                   |
| am08 | Code-to-Data 實戰改寫              | 挑一段現有 ABAP 報表邏輯 ( 內表迴圈 + 巢狀運算 ) 改寫成 AMDP 版本，前後對照可讀性/寫法差異；何時該下推、何時不該 ( 不是所有邏輯都適合 )                                                                                                                                                                                                                            | 對照 OOP op12 報表重構的定位                                             | 未出題                                                                                   |

| #                        | 主題                                                           | 內容重點                                                                     | 銜接前面課程                     | 狀態  |
|--------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------|----------------------------|-----|
| am09<br>( 期<br>末整<br>合 ) | 綜合實作：<br>分析型報表<br>用 AMDP<br>+ CDS<br>Table<br>Function<br>呈現 | 整合 am01~am08，一支完整報表用<br>AMDP 處理聚合統計、CDS Table<br>Function 包裝、ABAP 端只做呈現層 | 對照 REST rs09 期末整<br>合的收斂角色 | 未出題 |

不碰的範圍 ( 明確排除 )

RAP ( Behavior Definition/Service Binding 等 )、CDS View 的完整 Annotation 體系、HANA PlanViz 效能調校細節、SQLScript 進階特性 ( Table UDF、Scalar UDF、Graph 相關 ) ——這些留給有需要時另開課或另一階段。

出題工作流程 ( 比照 OOP/REST 課程 )

SAP 建物件 ( `sap_create_object` ; CDS Table Function 等 MCP 不支援的物件類型見 `.claude/rules/sap-adt-mcp.md` 的 ADT API workaround，需要時另外補充 AMDP/DDLS 專屬的建立方式 ) → `sap_set_source` 寫入 → 語法檢查 + 啟用 → 使用者用 ADT 或 SE38 實測 ( AMDP 無 SICF 需求，不需要瀏覽器測試 ) → 使用者驗收 → 快照 → 題目 md → `node tools/md2pdf.js src/ABAP_Training_AMDP` 產 PDF → 更新本 README 狀態 → commit。每批 2-3 題、使用者驗收後再繼續。